

Contents

Written $\neq$ Wired: Hunting Fake-Green in Agent Systems	1
Thesis	1
1. Taxonomy — four species of fake-green	1
2. Cases — all measured, July–August 2026	2
3. Doctrine — what actually works	3
4. Closing	4
Author’s note — AI contribution	5

Written $\neq$ Wired: Hunting Fake-Green in Agent Systems

Prof. Dr. Mustafa Melikoğlu · Yağız Deniz Altınbaş · Tayfun Tanrıöver

Preprint · 20 August 2026 · CC BY 4.0

On 19 August 2026, a push notification reported success and never fired. The whole failure was one shell line: a timeout-prefixed push whose printed `rc=0` belonged to the trailing echo, not to the push — and on that macOS box timeout isn’t installed, so the guarded command never ran at all. Exit zero. Green. Nothing sent. Fake-green fits in a single line, and if you run fleets of agents that *generate* lines like this on request, you are manufacturing it at machine speed.

Thesis

(*verbatim from spine — do not edit*)

A green light is a claim, not a fact. In agent systems the most expensive failures are not red — they are **fake-green**: the defense that was *written but never wired in*, the process that *exists but isn’t running*, the exit code 0 that *reports done without doing*, and the check that *measured the wrong surface*. The only cure is counterfactual verification: **a protection you have never watched fail has never been tested**.

1. Taxonomy — four species of fake-green

Fake-green is not one bug; it is a family, and the family members hide in different places. Naming them is the first defense, because you cannot grep for a thing you have not named.

1. **Written $\neq$ Wired** — the code exists, the call site doesn’t. The guard is written, reviewed, merged — and never invoked. An armory full of weapons nobody is carrying.
2. **Exists $\neq$ Running** — the daemon, hook, or alarm is installed on paper and absent from the scheduler. The design is real; the running system quietly disagrees with it.
3. **`rc=0` $\neq$ Done** — the pipeline swallows an inner failure and reports success up the chain. The work didn’t happen; the exit code says it did.
4. **Measured the wrong surface** — the verification reads the source, the log, or the claim instead of the live artifact. It checks that something *was written*, not that it *is true right now*.

The four share one property that makes them dangerous: each produces a green signal by a mechanism that is entirely local and entirely plausible. Nothing is lying on purpose. That is exactly why a passive reviewer misses them — and why the only reliable detector is an active one, described in §3.

2. Cases — all measured, July–August 2026

Each case below is a real incident from our own operations, dated to the month it occurred, told with the numbers we can re-derive from our ledgers and nothing more. They were chosen so that no case is retold from the sibling article on the same theme.

Case A ($rc=0 \neq \text{Done}$): the backup that swallowed its own failure

A nightly off-machine backup reported success for eighteen consecutive nights — 15 July through 2 August 2026 — while its actual copy step failed every single time. The credential the job used to reach the remote source had stopped being accepted, so the transfer never happened; but the surrounding legs — listing the archive, pruning old sets, writing the log — all kept succeeding, which is exactly why nothing looked wrong. Local copies sat silently frozen the whole time.

The mechanism was one misplaced line. The inner step's return code was captured into a variable and written to the log, and then the script ended with a hardcoded `exit 0` placed *after* that capture — the failure was recorded and then thrown away before it could reach the exit code the monitor watched.

The fix was to propagate the inner code ($\$RC$) instead of discarding it. But the fix is not the lesson; the counterfactual is. The very next run *screamed* — a visible 255, the error that had been there all along, now finally allowed to surface. A backup you have never watched fail is a backup you are only hoping works.

Case B (Written $\neq$ Wired $\rightarrow$ fail-closed pair): two fail-opens closed in one day

Two independent protections — a seat-call brake and a health-filter producer-pulse — both *looked* protective and both failed **open** when their precondition was absent. The brake went silently green the moment its measuring tool was disabled: written, merged, and in the one situation that mattered, not wired to bite. A brake that releases exactly when it cannot measure what it is braking on is not a brake; it is a green light with a brake-shaped label.

Both were closed on the same day — 4 August 2026 — by inverting the default to fail **closed**, and the closure shipped with 8 counterfactual tests, split 5 + 3. Five belong to the brake: they prove it now refuses to proceed when it cannot measure, using a dedicated exit code — $rc=6$, reserved for “could not measure” — so that not-measuring is itself a stop. The other three belong to the pulse, and its fix is a *different* mechanism: an input it cannot measure is scored as maximally stale, which trips the red rail; the script itself always exits 0 and signals through a flag file, and never touches $rc=6$. The two must not be conflated — pinning one code to both protections would be the very $rc=0 \neq \text{Done}$ confusion this article is about. (Those 8 tests are scoped to the closure itself; a separate follow-up repair on the same tool that day is a different count.)

Case C (Exists $\neq$ Running): the alarm that was “armed” in the notes

An operations ledger recorded that a critical re-arm alarm was “not found in any scheduler — stale note, must re-install.” A reasonable engineer reading that note would have spent the morning reinstalling a live alarm. Live measurement said the opposite: the alarm was loaded, and its last exit was 0. The note and the scheduler had *diverged*, and the note was the one that was wrong.

This is the whole disease in one incident, and it cuts both ways: the danger is not only “the notes say armed but it's dead,” but equally “the notes say dead but it's armed.” A diary of intentions is not a system state. Only the live surface — the platform's own service list, not the human record beside it — could arbitrate, and it did in seconds. Note that this case also demonstrates species 4: the near-mistake came from trusting a written claim over the running artifact. (Ledger: 19 August 2026, morning.)

Case D ($rc=0 \neq \text{Done}$, signature form): the 0-byte success

A model-CLI adapter returned `exit 0` and wrote zero bytes. This is the purest fake-green signature we have on file: `exit=0 + 0-byte`, now a banned-pattern check in our tooling notes.

We caught a fresh instance of it comparatively. The same prompt was dispatched to four sibling model backends in the same comparative run; one returned `rc=0`, wrote no error file, and produced 0 bytes of output, while the other three returned the correct answer. The fault was not in the wrapper — it was at a single endpoint, and it emitted **no error signal at all**. Only a check that inspects the *content* of the output could have caught it; a check that reads the exit code would have logged “success.” The failure was invisible to every signal except the bytes themselves. (Measured 19 August 2026.)

There is a sharper point buried in how we found it: we saw the empty return only because we were blind-testing a capability on its very first use. Had we simply started using it, we would have built work on top of a call that silently returns nothing — and never known. That is Doctrine 4 stated as a story: green must be earned, and a capability you have never watched fail is a capability you are trusting on faith.

Cameo (one sentence + link)

The purest **Written $\neq$ Wired** specimen we have — a deduplication defense that was one import away and still absent at the comparison site, later closed with mutation tests in which 3/3 mutants died — is told in full in the companion article, *The Turkish I Still Breaks Your Software (and Your AI Agents)* (Melikoğlu, Altınbaş & Tanrıöver, 2026), <https://doi.org/10.5281/zenodo.22018298> — cited here as a pointer, not retold.

3. Doctrine — what actually works

None of the following is novel as a technique; each has a mature discipline behind it. What is new is applying them to *defenses an agent generated on request*, at fleet scale, where the thing being verified is the guard itself.

1. **Counterfactual or it doesn’t exist.** Every protection ships with a test that removes or mutates it and watches the failure happen. If killing the defense does not break a test, the test was not testing the defense. This is precisely the thesis of **mutation testing**: seed a fault into the code, run the suite, and if no test fails, the mutant “survived” and the suite is inadequate — the discipline exists to answer “who tests the tests.” The idea is old: it traces to DeMillo, Lipton & Sayward, “Hints on Test Data Selection” (IEEE Computer 11(4):34–41, 1978, <https://doi.org/10.1109/C-M.1978.218136>) and is surveyed comprehensively by Jia & Harman (IEEE TSE 37(5):649–678, 2011, <https://doi.org/10.1109/TSE.2010.62>); the tool most engineers meet it through is PIT (<https://pitest.org/>), with a readable overview at https://en.wikipedia.org/wiki/Mutation_testing. The cameo above closed exactly this way — 3/3 mutants killed.
2. **Verify on the live surface.** The scheduler, the rendered artifact, the bytes on disk — never the note, the source, or the claim. Notes drift; surfaces don’t lie. Case C is this doctrine’s whole argument: the human record and the running system disagreed, and only the running system could settle it.
3. **Fail-closed as default posture.** “Could not measure” is a stop, not a pass. A protection whose failure mode is “wave it through” (Case B) is worse than no protection, because it also carries a green label. Inverting the default is what turned Case B’s two fail-opens into protections that bite — each in its own way: the brake now halts on a dedicated “could not measure” code (`rc=6`), while the health filter scores an unmeasurable input as maximally stale so it trips the red rail instead of passing.
4. **Green must be earned.** A check that has never been red is unproven; make it fail once on purpose — chaos-lite, one-shot, cheap. This is the local, low-cost cousin of **chaos engineering**, which builds

confidence by injecting real failures and observing whether the system withstands them (Principles of Chaos Engineering: <https://principlesofchaos.org/>; Netflix Chaos Monkey randomly terminates live instances to force resilience: <https://github.com/Netflix/chaosmonkey>). The honest difference is scope: chaos engineering runs continuous experiments against a production system for resilience; our counterfactual is a single deliberate break against one defense, run once before that defense is trusted. Same instinct, far smaller blast radius.

5. **Hunt the signatures.** Fake-green accumulates recognizable *signatures* — `exit-0 + 0-byte`; “armed in the notes”; `imported-but-never-called`; `rc` that belongs to the wrong command. Keep a ledger of them and `grep` for them in review. The “`imported-but-never-called`” signature is the exact target of **dead-code detection**: tools that statically flag unused imports, functions, and unreachable branches (Vulture for Python: <https://github.com/jendrikseipp/vulture>). It is not a toy concern — the canonical example of unreachable “dead” code shipping green is Apple’s 2014 `goto fail`; TLS bug (https://en.wikipedia.org/wiki/Unreachable_code). A team’s signature ledger is worth keeping as a literal table you can `grep` against in review:

Signature	Where to grep
<code>exit-0 + 0-byte</code>	a success path that produced no output
<code>armed-in-notes</code>	any “installed / armed” claim not confirmed against the live scheduler
<code>imported-never-called</code>	dead-code / unused-import linter at the call site
<code>rc</code> belongs to the wrong command	<code>timeout/wrapper</code> prefixes where <code>\$?</code> is not the guarded command’s

Where this sits in the literature: mutation testing, chaos engineering, and dead-code detection are established fields, and we are not reinventing them. The claim of this piece is narrower and, we think, newly urgent — that these instincts have to be turned on *generated defenses*, the guards an agent writes and reports as done, because that is where fake-green is now produced fastest.

4. Closing

Fake-green is worse than red because red interrupts you and fake-green compounds. A red build stops the line; a fake-green one lets you keep building on a floor that isn’t there, and the cost arrives later, larger, and detached from its cause.

Agent fleets multiply the risk in a specific way. Agents *generate* defenses on request, and generated code inherits the same disease at machine speed: an agent will happily write the guard, report success, and never wire it in — Case B and the cameo are both that story, one by hand, one machine-made. Every case in this piece was a signal that existed but did not propagate, or a claim that was never checked against the artifact it described. The 0-byte success (Case D) is the limit form: a success with no work inside it, invisible to every signal except the bytes.

The teams that survive are not the ones with the most checks. They are the ones whose checks have all been watched failing at least once.

What we do not claim. There is no silver bullet here. Counterfactual tests cost real engineering time — you are paying to break things you built, on purpose, before you need them. And we are not done finding species: this taxonomy has four entries because four are what we have measured, not because the family is closed. We expect to add a fifth, and we would rather say so than pretend the list is complete — which would itself be a kind of fake-green.

Author’s note — AI contribution

This article was produced inside an AI-agent ecosystem, and it is about that ecosystem’s own failures. In keeping with scientific-publishing practice, the named authors of record are the humans who built the ecosystem; the AI contribution is disclosed here openly.

The thesis and section structure were set by the ecosystem’s coordinating AI identity. The body was written by a second AI identity working strictly on that structure, under an apprentice rule: it deepens the given spine and does not mint claims of its own. A third AI identity ran the verification gates — fetching and checking every external citation against the work it claims to cite, measuring each case’s numbers back to the primary record that recorded them, and screening the text before release.

Nothing here is hypothetical. Every case is an incident from our own operations, dated, and re-derived from the ledger that recorded it at the time. One number in an earlier draft was wrong and was caught by exactly the discipline this article argues for; that correction is why the figure you are reading is the measured one.

Before release the draft was reviewed by an adversarial panel of judge models from families other than the one that wrote it — a producer-is-not-the-judge rule the ecosystem applies to itself. The panel confirmed the piece and returned a defect list, one item of which turned out to be the panel’s own error; that item was adjudicated against the primary record rather than accepted on the panel’s authority. Both directions of that exchange are, we think, the point.